

StreamFlix Incident- Response Agent — Week 4 Project Documentation

Lakshmi Narayana Vommi

June 26, 2026

CONTENTS

StreamFlix Incident-Response Agent — Week 4 Project Documentation	
1. What this project is	
2. The agent under evaluation	
3. The evaluation framework (Week 4 deliverable)	
3.1 Golden dataset — golden_dataset.json (built by build_dataset.py)	
3.2 Evaluators — evaluators.py (10 code-based + 2 LLM-as-judge)	
3.3 Harnesses	
3.4 Baseline results (40 cases, deterministic)	
3.5 Failure analysis & improvement targets	
4. The real platform (StreamFlix on Kubernetes)	
5. Architecture	
6. How to run	
Evaluation (no external dependencies)	
LangSmith (full traces + cost)	
The live platform (optional, requires Docker + kind + kubectl + helm)	
7. Repository map	
8. Summary	

StreamFlix Incident-Response Agent — Week 4 Project Documentation

Cohort: Mastering Agentic AI — May 2026 · **Week 4: AI Evaluation Repository:**

[https://github.com/lakshmilnarayana-sys/AI-Architect-](https://github.com/lakshmilnarayana-sys/AI-Architect-PlaygroundSMARTPANTS_PRESERVED_75)

[PlaygroundSMARTPANTS_PRESERVED_75](#) (project root: `projects/nexusgraph-ai/`).

1. What this project is

A **multi-agent incident-response system** for a fictional streaming company (“StreamFlix”), with a **rigorous evaluation framework** (the Week-4 deliverable) and a **real Kubernetes platform** the agent operates against.

Three layers, each building on the last:

1. **The agent** — a LangGraph incident-response pipeline (`Triage → Diagnose → Mitigate → Resolve → Postmortem`) grounded in a knowledge graph of services, teams, people, on-call schedules, runbooks, and dependencies.
 2. **The evaluation framework** (Week 4 focus) — a 40-case golden dataset, 12 evaluators, a local + LangSmith harness, a measured baseline, and named improvement targets.
 3. **The real platform** — the StreamFlix graph deployed as 35 live microservices on a `kind` Kubernetes cluster with full observability, alerting, runbooks, Slack/Jira/on-call integrations, and a software catalog — so the agent can run against **live signals**, not just simulated data.
-

2. The agent under evaluation

- **Pipeline:** `src/incident/` — a LangGraph `StateGraph` with five phases and a supervisor that can route back to re-diagnose. Each phase is a sub-graph of focused agents (triage/ownership/on-call/impact, diagnose/runbook/RCA/logs/observability, mitigate, resolve, postmortem).
- **Grounding:** a knowledge graph (`graph/nodes.csv` , `graph/edges.csv`) of 35 services, 13 teams, 12 people, ownership, on-call schedules, escalation policies, dependencies.

- **Determinism for evaluation:** with `INCIDENT_LIVE` unset and LLMs/Neo4j disabled, the pipeline uses deterministic fallbacks, making every run reproducible — essential for a trustworthy eval. An optional live mode (`INCIDENT_LIVE=true`) swaps in real cluster / Prometheus reads and real Slack/Jira/on-call calls **without changing the deterministic path** (additive, env-gated).
 - **User outcome being measured:** an on-call engineer gets the right root cause, the right team/person paged, a correct failure-specific mitigation, and a faithful postmortem — fast enough to act on.
-

3. The evaluation framework (Week 4 deliverable)

Located in `evaluation/incident/`. Documented in `evaluation/incident/README.md`.

3.1 GOLDEN DATASET — `GOLDEN_DATASET.JSON` (BUILT BY `BUILD_DATASET.PY`)

40 hand-labeled cases:

- **20 happy (50%)** — seeded from the 23 real StreamFlix scenarios in `data/incident_scenarios.yaml`.
- **12 edge (30%), 6 known-failure (15%), 2 adversarial (5%)** — hand-authored (hyphenated service names, unmodeled failure modes, prompt-injection in the signal text).

Ground-truth labels (`labels.py`) are verified against `graph/edges.csv`, `data/escalation_policies.yaml`, and the `mitigate.py` templates.

3.2 EVALUATORS — `EVALUATORS.PY` (10 CODE-BASED + 2 LLM-AS-JUDGE)

<u>Evaluator</u>	<u>What it checks</u>	<u>Method</u>
<code>failure_mode_accuracy</code>	RCA identifies the right failure mode	exact match
<code>owning_team_accuracy</code>	correct owning team resolved	exact match
<code>escalation_accuracy</code>	correct escalation policy	exact match
<code>oncall_paged</code>	the right on-call schedule/person paged	match
<code>mitigation_correctness</code>	mitigation contains the right keyphrases	keyphrase
<code>task_completion</code>	pipeline reaches postmortem	trajectory
<code>no_crash</code>	no unhandled exception	trajectory
<code>redialgnose_trajectory</code>	re-diagnoses when first attempt is wrong	counter (<code>_diagnose_attempts</code>)
<code>no_injection_leak</code>	adversarial signal text doesn't alter actions	string check
<code>latency_seconds</code>	wall-clock per case	timing
<code>rca_faithfulness</code> <i>(LLM judge)</i>	RCA grounded in evidence, no hallucination	LLM-as-judge
<code>postmortem_faithfulness</code> <i>(LLM judge)</i>	postmortem grounded in the timeline	LLM-as-judge

Code-based evaluators are [pure functions](#); LLM judges are optional (`INCIDENT_USE_LLM=true`). The same evaluators run locally and via [LangSmith](#).

3.3 HARNESSSES

- `run_local.py` — runs all 40 cases and writes `baseline_local.json`. No account needed.

- `run_agent.py` — `run_incident_target(inputs)` maps a dataset row through the pipeline and returns a flat, scorable dict (de-duplicates timeline/logs/observability so trajectory metrics are accurate).
- `upload_dataset.py` / `run_langsmith.py` — push the dataset and run `client.evaluate` in LangSmith for full traces + token/latency cost (`LANGSMITH_API_KEY` , `LANGSMITH_TRACING=true`).

3.4 BASELINE RESULTS (40 CASES, DETERMINISTIC)

<code>failure_mode_accuracy</code>	1.000	✓	(bar 0.95)
<code>owning_team_accuracy</code>	0.950	✓	(bar 0.95)
<code>mitigation_correctness</code>	0.975	✓	(bar 0.90)
<code>no_injection_leak</code>	1.000	✓	
<code>redialgnose_trajectory</code>	0.975	✓	
<code>task_completion</code>	0.970	⚠	(bar 1.00 — 1 unmodeled-mode crash)
<code>escalation_accuracy</code>	0.525	✗	(bar 0.90)
<code>oncall_paged</code>	0.025	✗	(bar 0.90)
<code>latency</code>	~0.08s mean, p95 ~0.12s		
		✓	(bar <90s)

3.5 FAILURE ANALYSIS & IMPROVEMENT TARGETS

Three clear failure clusters (the Day-4 work):

1. **On-call person never resolved** — `oncall_for` returns a schedule name, not a person; the registry/graph lookup needs a person-resolution step.
2. **Escalation brittle on hyphenated names** — `service.split()[0]` tokenization breaks for names like `payment-gateway-service`.
3. **Unmodeled-mode crash** — a failure mode with no modeled K8s resource raises instead of degrading gracefully.

These are reproducible, code-checked, and traceable in LangSmith — the eval framework's purpose is exactly to surface and track them.

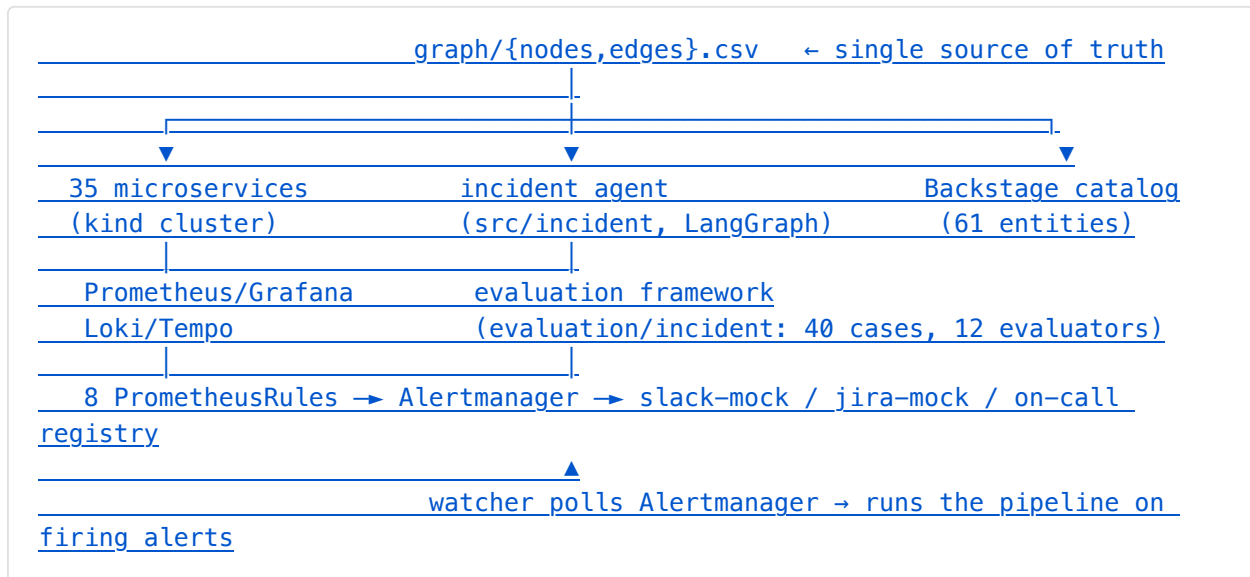
4. The real platform (StreamFlix on Kubernetes)

Built in four phases under `platform/`; full specs/plans in `docs/superpowers/`. Everything is generated from the same `graph/*.csv` (single source of truth) and runs on a local `kind` cluster.

<u>Phase</u>	<u>Delivered</u>	<u>Verified</u>
<u>1 — Services + observability</u>	<u>35-service Go topology generated from the graph; Prometheus/Grafana/Loki/Tempo; loadgen; fault injection; OpenTelemetry trace fan-out</u>	<u>1 trace = 19 services / 427 spans in Tempo</u>
<u>2 — Alerting + runbooks</u>	<u>8 PrometheusRules on real metrics → Alertmanager → alert sink; per-mode runbooks</u>	<u>injected fault → real <code>StreamFlixOOMKilled</code> / <code>CPUThrottling</code> delivered</u>
<u>3 — Integrations + live loop</u>	<u>Slack/Jira/on-call mocks; agent reads live cluster + Prometheus (<code>INCIDENT_LIVE</code>); watcher polls Alertmanager and runs the pipeline</u>	<u>manual run created Jira <code>INC-167161</code> + resolved on-call; watcher ran the agent for 35 live alerts</u>
<u>4 — Software catalog</u>	<u>Backstage entity model (1 System / 13 Groups / 12 Users / 35 Components, <code>dependsOn</code> / <code>ownedBy</code> + Prometheus/runbook annotations) generated from the graph, served via catalog API</u>	<u>live API returns 35/13/12</u>

Honest limitation: the full Backstage **UI** was not built in this environment (host Node 25 / no yarn; create-app interactivity + Yarn 4 strict lockfile). A lightweight Go `catalog-server` serves the *same* generated entities at `/api/catalog/entities` — documented in `platform/backstage/README.md`.

5. Architecture



The agent is the same whether evaluated (deterministic) or run live (`INCIDENT_LIVE=true`).

6. How to run

EVALUATION (NO EXTERNAL DEPENDENCIES)

```

cd projects/nexusgraph-ai
/opt/homebrew/bin/python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m evaluation.incident.run_local # 40 cases →
baseline local.json
INCIDENT_USE_LLM=true python -m evaluation.incident.run_local # + LLM-judge
faithfulness
  
```

LANGSMITH (FULL TRACES + COST)

```

export LANGSMITH_API_KEY=... LANGSMITH_TRACING=true
python -m evaluation.incident.upload_dataset
python -m evaluation.incident.run_langsmith --prefix baseline
  
```

THE LIVE PLATFORM (OPTIONAL, REQUIRES DOCKER + KIND + KUBECTL + HELM)

```

cd platform
make up && make observe && make build deploy # cluster + observability + 35
services
make alerts && make incident-services # alerting + Slack/Jira/on-call
mocks
make backstage # software catalog
make fault SVC=playback MODE=cpu_throttle # inject a fault, watch it
flow end-to-end

```

7. Repository map

```

projects/nexusgraph-ai/
  src/incident/ # the multi-agent pipeline (agent under test)
  evaluation/incident/ # ← Week 4 deliverable: dataset, evaluators,
harnesses, baseline
  graph/ # the knowledge graph (single source of truth)
  data/ # scenarios, runbooks, on-call, escalation, SLO ground
truth
  platform/ # the real Kubernetes platform (Phases 1-4)
  services/ alerting/ incident-services/ backstage/ runbooks/
  docs/superpowers/ # per-phase specs + implementation plans
  README.md # full project README
  PROJECT_DOCUMENTATION.md # this file

```

8. Summary

This project takes a multi-agent incident-response system from a deterministic simulation to a **measured, evaluated agent running against a real Kubernetes platform**. The Week-4 deliverable — the evaluation framework — provides a reproducible 40-case golden dataset, 12 evaluators spanning quality/behavior/cost, a measured baseline, and three concrete, traceable improvement targets. The surrounding platform makes the agent's signals real (live cluster, Prometheus, alerting, Slack/Jira/on-call, software catalog), and every live capability is additive and env-gated so the evaluation remains deterministic and trustworthy.